

## 11 — L'oggetto incompleto

**La domanda a cui risponde:** se tre punti diversi del codice costruiscono “un mazzo”, chi garantisce che siano davvero la stessa cosa? E perché in Java questa domanda non te la poni quasi mai, mentre in JavaScript ti si presenta sotto forma di `Cannot read properties of undefined`?

### Il fatto

Il 26 luglio 2026, premere  su un mazzo costruito a mano e salvato produceva due errori diversi a distanza di dieci minuti:

```
undefined is not an object (evaluating 'carta.idSet')
Cannot read properties of undefined (reading 'pokemon')
```

Sembrano due bug. Sono lo stesso bug, contato due volte: **un oggetto che in alcuni punti del programma ha certi campi e in altri no**.

Nel progetto un “mazzo” nasce in tre posti:

| Dove               | Chi lo costruisce                                                            | Cosa ci mette                                          |
|--------------------|------------------------------------------------------------------------------|--------------------------------------------------------|
| generazione        | <code>src/engine/generazione.js</code>                                       | nome, carte, totale, composizione, tipi                |
| costruttore a mano | <code>mazzoCorrente()</code> in <code>src/app/vista-personalizzato.js</code> | nome, carte, totale, tipi — <b>niente</b> composizione |
| rilettura da disco | <code>idrataPiano()</code> in <code>src/data/mazzi-salvati.js</code>         | quello che era stato salvato                           |

Finché i mazzi fatti a mano restavano nella loro schermata, la differenza non si vedeva: quella schermata usa solo `carte` e `totale`. Il guaio arriva quando il mazzo attraversa il confine — si salva, si riapre nell'altra sezione, e finisce dentro funzioni scritte per i mazzi *generati*.

### Perché in Java non succede

In Java scriveresti una classe:

```
public final class Mazzo {
    private final List<Voce> carte;
    private final Composizione composizione;    // mai null

    public Mazzo(List<Voce> carte) {
        this.carte = List.copyOf(carte);
        this.composizione = Composizione.conta(carte);    // l'invariante
    }
}
```

Il costruttore è un **collo di bottiglia**: non esiste modo di ottenere un `Mazzo` senza passare di lì, quindi non esiste un `Mazzo` senza `composizione`. È l'idea di *invariante di classe* — una proprietà vera all'uscita dal costruttore e mantenuta da ogni metodo.

In JavaScript un oggetto letterale non ha collo di bottiglia. `{ nome, carte, totale }` è un mazzo valido quanto uno con dodici campi: nessuno lo controlla, perché non c'è nessun tipo da soddisfare. La forma dell'oggetto è **una convenzione fra i punti del codice che se lo passano**, e le convenzioni non scritte divergono appena due persone (o due rami) lavorano in parallelo.

Non è che JavaScript sia peggio: è che l'invariante, non avendo un posto dove abitare, devi metterla tu da qualche parte esplicitamente.

## I due modi in cui si rompe

### 1. Il campo che non c'è mai stato

CostruttoreMazzo.carte (in src/ui/costruttore-mazzo/costruttore-mazzo.js) restituiva le carte così:

```
.map(([, k, quantita]) => ({ carta: per.get(k)?.carta, quantita })))
```

Sembra corretto. Ma voce.carta viene dal *dataset*, e nel dataset la carta **non contiene** idSet e numero: quei due campi stanno sulla riga di collezione che la avvolge, non dentro la carta. Il costruttore restituiva quindi carte anonime, e ogni funzione che identifica una carta con idSet + numero — cioè tutte — lavorava su undefined/undefined.

La correzione ricopia i due campi entrando:

```
carta: { ...voce.carta, idSet: voce.carta.idSet ?? voce.idSet,
        numero: voce.carta.numero ?? voce.numero },
```

?? e non ||: un numero uguale a 0 o "" è un numero, e || lo scarterebbe. È la stessa distinzione fra “assente” e “vuoto” del documento 08.

### 2. Il campo che il chiamante aggiorna invece di calcolare

aggiungiAlMazzo() in src/engine/mazzo.js fa così:

```
mazzo.composizione[campo] += aggiungibili;
```

**Aggiorna**, non ricalcola. È una scelta ragionevole — ricontare tutte le carte a ogni singola aggiunta sarebbe O(n) per niente — ma trasforma composizione da *risultato* a **precondizione**: la funzione pretende che esista già. Su un mazzo senza quel campo il += diventa undefined['pokemon'] += 1, ed è il secondo errore.

Qui c'è una lezione generale: **ogni ottimizzazione incrementale crea una precondizione**. Nel momento in cui smetti di derivare un valore e cominci a mantenerlo, qualcuno deve garantire che il valore di partenza ci sia e sia giusto. In Java quel qualcuno è il costruttore; qui bisogna nominarlo.

## La correzione: un posto solo che sa contare

Il rimedio non è stato spargere if (!mazzo.composizione) nei punti d'uso — sarebbe stato *nascondere* il problema in cinque posti invece di risolverlo in uno. È stata una funzione esportata dal modulo che possiede il concetto:

```
// src/engine/mazzo.js
export function contaComposizione(carte) {
  const conti = { pokemon: 0, energie: 0, allenatori: 0 };
  for (const voce of carte ?? []) {
    const campo = CAMPO[voce.carta?.categoria];
    if (campo) conti[campo] += voce.quantita ?? 0;
  }
  return conti;
}
```

Chiamata in **due** posti, che sono i due confini da cui entrano mazzi stranieri:

1. mazzoCorrente() — dove nasce un mazzo fatto a mano;
2. idrataPiano() — dove rientra un mazzo salvato prima che il punto 1 esistesse.

Il secondo è quello interessante. I dati già sul disco degli utenti non si possono correggere retroattivamente: nessuno può lanciare una migrazione sul telefono di casa. Quindi la rilettura è il posto giusto per ricostruire ciò che manca — è già la funzione che “sviluppa la fotografia”, e aggiungere un campo derivabile è esattamente il suo mestiere:

```
composizione: mazzo.composizione ?? contaComposizione(carte),
```

Di nuovo ??: chi la composizione ce l'ha se la tiene, e non gli viene ricalcolata sotto i piedi.

## La regola pratica

Un campo **derivabile** dai dati che hai si ricostruisce al confine di lettura. Un campo **non derivabile** (`idSet`, `numero`: informazione che se perdi non torna) va garantito al confine di *scrittura*, perché dopo è troppo tardi.

È la ragione per cui i due bug si correggono in due posti diversi pur avendo la stessa forma.

## Perché i test non l'avevano preso

I test del motore erano verdi, tutti e 262. Costruivano i mazzi così:

```
const mazzo = (voci) => ({ nome: 'Prova', carte: voci });
```

Cioè con un **quarto** costruttore di mazzi, quello dei test, che come gli altri tre metteva campi diversi. Un test che si fabbrica il proprio input non può scoprire che l'input vero ha un'altra forma: verifica la funzione contro la propria idea della funzione.

Questo è il limite dei test unitari su codice non tipizzato, ed è utile saperlo: un test verde dimostra che *quel* dato produce *quel* risultato, non che il dato somigli a quello che arriva davvero. Il difetto è comparso solo premendo un pulsante nel browser, sulla catena completa costruttore → salvataggio → rilettura → sostituzione.

I test aggiunti dopo, infatti, non provano una funzione: provano **una transizione**. `tests/mazzi-salvati.test.js` costruisce un record *come stava su disco prima del fix* e verifica che rileggendolo l'invariante ci sia:

```
const mazzo = idrataPiano(record).mazzi[0];  
assert.deepEqual(mazzo.composizione, { pokemon: 3, energie: 2, allenatori: 1 });
```

## Esercizi

1. **Trova il quarto costruttore.** In `tests/completa-mazzo.test.js` la funzione `mazzo()` produce oggetti senza `composizione` né `totale`. Scrivi un test che passi uno di quei mazzi a `aggiungiAlMazzo()` e osserva l'errore. Poi decidi: è meglio correggere l'helper dei test, o è meglio che `aggiungiAlMazzo()` si difenda? Argomenta — non c'è una risposta sola, ma ce n'è una coerente con la regola pratica qui sopra.
2. **Deriva o garantisci.** Per ciascuno di questi campi di un mazzo salvato di', e spiega perché: `totale`, `tipi`, `forza`, `nome`, `motivo` di una voce proxy. Quali si possono ricostruire in `idrataPiano()` e quali no? (Suggerimento: `forza` è il caso più sottile — il commento in `istantanea()` spiega perché *non* va ricalcolata anche se si potrebbe.)
3. **L'invariante in JS.** Riscrivi `mazzoCorrente()` come una class `Mazzo` con un costruttore che garantisce `composizione` e `totale`. Poi chiediti perché il progetto **non** l'ha fatto: cosa succederebbe a un'istanza di classe passata a `structuredClone()` per finire in `IndexedDB`? (È la stessa ragione per cui `src/engine/` lavora su oggetti nudi. La risposta sta nel documento 02.)